Razumijevanje umjetne inteligencije kroz primjenu u ekonomiji

Maks Gregorić, 4.B

Maks.Gregoric@skole.hr

Makgregoric@gmail.com

Sažetak

Ovaj rad istražuje umjetnu inteligenciju i duboke neuronske mreže, specifično proučava LSTM (Long Short-Term Memory) modele i njihovu primjenu u ekonomiji, konkretno pri predviđanju cijena nekretnina u Sjedinjenim Američkim Državama. Istraživanje počinje s detaljnim teorijskim okvirom koji daje osnovno znanje čitatelju za ostatak rada. Metodološki dio rada, koji se bavi samom izgradnjom LSTM modela, počinje prikupljanjem i uređivanjem podataka, zatim prikazuje proces građenja arhitekture modela te napokon treniranjem modela. Rezultati pokazuju da sagrađen model uspješno predviđa opće trendove, ali ima problem s konkretnim vrijednostima te poteškoće u periodima nagloga rasta tržišta.

Ključne riječi: Umjetna inteligencija, neuronska mreža, LSTM, nekretnina, ekonomija

Summary

This paper explores artificial intelligence and deep neural networks, focusing on LSTM (Long Short-Term Memory) models and their application in economics, more precisely in predicting real estate prices in the United States of America. The research begins with a detailed theoretical framework that provides background knowledge. The methodological part of the paper deals with the building of the LSTM model itself and begins with the collection and preprocessing of data, then shows the process of building the model architecture, and finally the training of the model. The results show that the built model successfully predicts general trends, but has a problem with specific values, and difficulties in periods of sudden market growth.

Keywords: Artificial intelligence, neural network, LSTM, real estate, economics

Uvod

Umjetna inteligencija je grana računalstva koja proučava kako napraviti računalne sisteme koji imaju sposobnosti obavljati zadatke koje inače povezujemo s ljudskom inteligencijom (Russell & Norvig, 2021). Strojno učenje je podskup umjetne inteligencije i podrazumijeva stvaranje algoritama i modela koji imaju sposobnosti „naučiti“ funkcije iz nekog skupa podataka. Duboko učenje je specijalizirani podskup strojnog učenja koji koristi duboke neuronske mreže (s više slojeva) za modeliranje složenih funkcija (Kelleher, 2019).

Korištenje već spomenutih algoritama za analizu podataka, predviđanje budućih trendova i optimizaciju određenih procesa vrlo je korisno implementirati u većinu sektora, ali možda je najkorisnije u ekonomiji. UI je koristan alat za prognozu ekonomskih pokazatelja, uključujući cijenu nekretnina, BDP-a, CPI-a i drugih brojki.

Glavni problem umjetne inteligencije u ekonomiji je što su tržišta jako kompleksni sistemi koji ovise o bezbroj varijabli. Njih se sve može riješiti poboljšavanjem efikasnosti modela te razvojem računalne snage, no veći problem je kako kvantificirati neke od brojnih varijabli kako bi to računalo moglo interpretirati. Npr. geo-političko stanje svijeta je možda najbitniji faktor kod predviđanja trendova, ali iako to ljudski mozak može interpretirati i zaključiti kako će utjecati na neke mikroekonomska područja, za računalo je to znatno teže.

Osnove umjetne inteligencije i LSTM mreža

Što je umjetna inteligencija i neuronska mreža

Umjetna inteligencija je računalni program koji nastoji odrediti matematičku funkciju koja najbolje replicira određene pojave. Ovo se najčešće ostvaruje s pomoću strukture koja je labavo zasnovana na strukturi ljudskog mozga. Ova struktura se zove neuronska mreža.

Neuronska mreža sastoji se od povezanih jedinica koji se nazivaju neuroni. Neuroni su međusobno povezani rubovima, kao što su stvarni neuroni povezani sinapsama (Hardesty, 2017). Svaki neuron prima izlaze prijašnjih spojenih neurona kao ulaz, zatim ih preradi. Ovaj prerađeni broj postaje vrijednost tog neurona te šalje svoju vrijednost dalje, ostalim spojenim neuronima. Za svaku vezu neurona postoji težina ulaza, broj koji diktira koliko je taj specifični ulazni podatak bitan, i pristranost ulaza koji je kao slobodni koeficijent koji omogućuje fleksibilnost pri modeliranju. Ove vrijednosti nazivaju se parametri. Da bi se odredila izlazna vrijednost neurona ponderirana suma ulaznih vrijednosti i pristranosti se provodi kroz aktivacijsku funkciju[1]. Izlaz neurona se dakle može izraziti sljedećom formulom:

$$y = f(\sum_{i=1}^{n}(t_i * x_i) + p)$$

pri čemu su:
y – izlazna vrijednost neurona
f – aktivacijska funkcija
t – težina ulaza
x – ulazna vrijednost

Neuroni su tipično organizirani u slojeve. Različiti slojevi najčešće obavljaju različite transformacije nad ulaznim vrijednostima (Bishop, 2006). Prvi sloj naziva se i ulazni sloj jer se kroz njega upisuju podaci u mrežu, tipično ulazni sloj ima broj neurona koliko i svojstva imaju podaci nad kojima će mreža funkcionirati[2]. Zadnji sloj naziva se i izlazni sloj jer nam taj sloj daje izlaznu vrijednost, tipično izlazni sloj ima onoliko neurona koliko ima i mogućih odgovora[3]. Između ulaznog i izlaznog sloja nalaze se, takozvani, skriveni slojevi koji zapravo transformiraju naše ulazne podatke u nešto iz čega će naš izlazni sloj moći predvidjeti odgovor. Broj skrivenih

[1] Funkcija koja mapira sumu na određeni interval po određenom pravilu (ovisi od funkcije do funkcije).
[2] Npr. ako mreža treba prepoznati što je u slici, broj ulaznih neurona odgovarat će broju piksela na slikama.
[3] Ako mreža treba prepoznati koja je znamenka nacrtana u slici, imat će 10 neurona u izlaznom sloju.

slojeva je teže za odlučiti. On ovisi o kompleksnosti problema i količini podataka, no ne postoji točno pravilo za odrediti optimalni broj slojeva. Također, ne postoji strogo pravilo za broj neurona po skrivenom sloju, no ima nekih pravila koja se koriste za određene vrste neuronske mreže. Manje popularan sloj u modernim neuronskim mrežama, ali i dalje vrijedan spomena, jest sloj izbacivanja, koji nasumično postavlja vrijednost nula nekim neuronima iz prijašnjeg sloja (Srivastava i dr., 2014). Ovo je naizgled suludo, ali je korisno kako bi se smanjila vjerojatnost prenaučenosti modela[4]. Vrijednosti koje nisu podešene su povećane kako bi ukupna suma ostala ista.

Treniranje modela

Nakon što se odredi arhitektura neuronske mreže može se početi s treniranjem. Treniranje modela je proces optimizacije parametara neuronske mreže kako bi se smanjila razlika između izlazne vrijednosti modela i stvarnog rješenja (Kelleher, 2019). Treniranje ili „učenje“ mreže je kombinacija par funkcija/algoritama, obično su to: širenje unatrag (*backpropagation*), optimizacijski algoritam i funkcija gubitaka (*loss/cost function*).

Najprije su svi parametri mreže postavljeni nasumično jer se još ne zna kako ih se treba postaviti da se dobiju točna rješenja. Logično je, kada se neuronsku mrežu pokrene i provede set podataka, a nasumično su joj postavljeni parametri, da će davati vrlo loše rezultate (Russell & Norvig, 2021).

Funkcija gubitaka točno prikazuje koliko je precizan model tako što mjeri koliko je pogrešan model bio u predviđanju, te je cilj treniranja modela minimalizirati ovu funkciju, a budući da je gubitak modela ovisan o parametrima, isti se trebaju podesiti.

Širenje unatrag (*backpropagation*) je naziv za metodu računanja gradijenta funkcije grešaka s odnosom na svaku težinu veze u neuronskoj mreži. Ona je funkcija koja diktira koliko svaka težina pridonosi ukupnoj grešci.

Način na koji se ažuriraju veze neurona ovisno o gradijentu zove se optimizacijski algoritam. On je vođen gradijentom i stopom učenja. Optimizacijski algoritam funkcionira tako da se koristeći gradijente, izračunate širenjem unatrag, podesila svaka težina. Tipično se nova težina računa ovom formulom

[4] Ovo je kada se model prejako poravna s malim setom dostupnih podataka te time loše prati trendove.

$$t_{nova} = t_{stara} - \alpha \frac{\partial G}{\partial t}$$

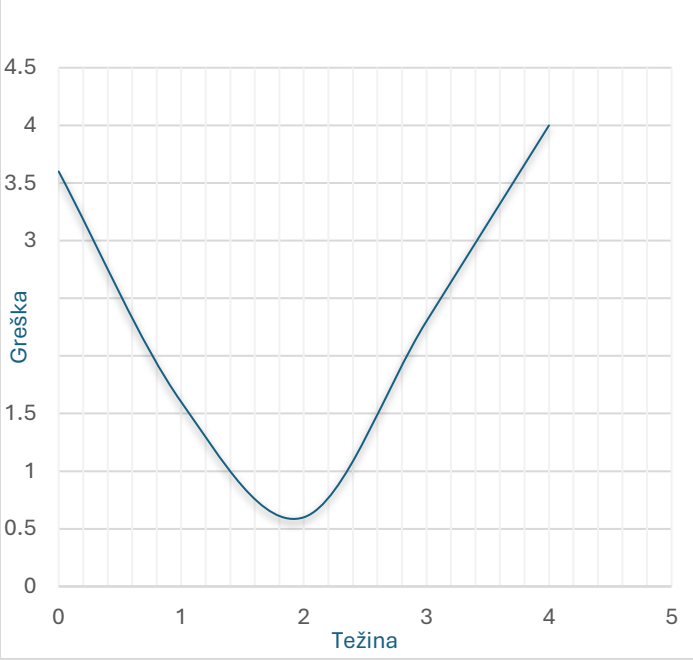
Pri čemu su:

t_{nova}, t_{stara} – nova težina i stara težina

α – stopa učenja (koliki „korak“ uzima algoritam, veći korak brže uči, ali možda prijeđe minimum, manji korak i model sporije uči, ali preciznije)

$\partial G/\partial t$[5] – gradijent greške s odnosom na težinu

Optimizacijski algoritam, ukratko, traži minimum grafa funkcije grešaka u odnosu na težine. Različiti optimizacijski algoritmi razlikuju se u tome kako, ako uopće, prilagođavaju stopu učenja.



Slika 1 Graf jednostavne/dvodimenzionalne funkcije grešaka

Dakle, treniranje je postupak slanja podataka mreži koja se onda svaki put malo podesi koristeći razne algoritme i funkcije. Za kvalitetno istrenirati model potrebno je jako puno informacija i jako dobro prilagođeni podaci.

[5] Djelomična derivacija – derivacija neke varijable u više dimenzija.

Vremenske serije, povratne neuronske mreže i LSTM model

Vremenska serija je serija podataka smještena u kronološkom redoslijedu. Najčešće, je vremenska serija niz podataka snimljenih u uzastopnim i jednakim vremenskim intervalima. Primjer vremenske serije je dnevna cijena neke dionice na burzi. Predviđanje vremenskih serija je korištenje modela da predvidi buduće vrijednosti, tako što bazira predviđanja na prijašnje zapaženim podacima.

Povratne neuronske mreže

Povratna (*Recurrent*) neuronska mreža (RNN) je vrsta neuronskih mreža kod kojih se izlaz neurona u nekom trenutku šalje nazad kao ulazni podatak mreže u idućem trenutku. Ova arhitektura modela omogućuje mreži da shvati uzorak u sekvencama. RNN-ovi su bili dizajnirani kako bi se procesirali sekvencijski podaci (vremenske serije, tekstovi , ...), kod kojih je redoslijed podataka bitan.

Kod RNN-ova se koristi povratna jedinica koja zadrži „skriveno stanje“. Ovo je vrsta memorije koja kroz svaku iteraciju mreže pamti prijašnje stanje (Sherstinsky, 2020). Problem kod ovih arhitektura je to da, pri treniranju, gradijenti između prijašnjih i kasnijih slojeva postaju sve manji i manji. Ovaj problem se naziva problem nestajućeg gradijenta (*Vanishing gradient problem*).

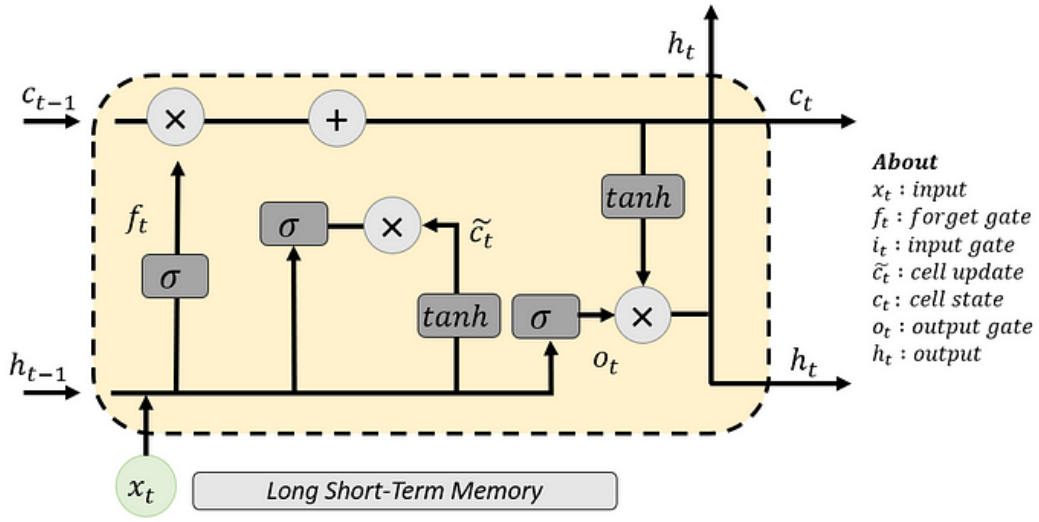
Problem nestajućeg gradijenta

Problem nestajućeg gradijenta je veliki problem jer se raniji slojevi jako malo, ili uopće ne ažuriraju tijekom širenja unatrag. Zbog toga razvoj mreže jako uspori ili čak stagnira. Veliku ulogu kod ovog problema igra aktivacijska funkcija jer, zbog njene ne-linearne prirode, za jako velike i jako male ulazne podatke gradijenti se približavaju nuli (Hochreiter, 1998). Algoritam širenja unatrag, koji računa gradijente funkcije grešaka zatim množi ove jako sitne gradijente pa rezultat konvergira nuli.

LSTM modeli

LSTM (Long Short-Term Memory) modeli su vrste RNN-ova koji su napravljeni kako bi se riješio problem nestajućeg gradijenta. LSTM modeli se razlikuju od ostalih modela jer se LSTM jedinica, koja služi umjesto klasičnih neurona, ima 3 vrata (*gate*), ulazna, izlazna i zaboravna (Sherstinsky, 2020). Pojednostavljeno, jedinica može pamtiti vrijednosti proizvoljno, a vrata reguliraju tok podataka kroz jedinicu. Zaboravna vrata odlučuju koje podatke će jedinica izbrisati iz prethodnog stanja. Ovo postiže tako što mapira prethodno stanje i trenutačni ulaz na interval od 0 do 1 te se zaokruži rezultat. Ako je rezultat 1 onda se sprema podatak, a ako je 0 onda se

briše. Ulazna vrata odlučuju koji podatci će se spremiti u trenutačno stanje jedinice istim sistemom kao što i zaboravna vrata odlučuju koje podatke će izbrisati. Izlazna vrata odlučuju koji podaci su „poželjni“ da se šalju dalje, ako se poželjni podaci šalju dalje LSTM mreža će biti uspješna.



Slika 2 Unutrašnjost LSTM jedinice

Dizajn i implementacija LSTM modela

Skupljanje podataka

Prije izgradnje modela najbitnije je sakupiti mnogobrojne i kvalitetne podatke. Kako bi se ovo postiglo potrebno je utvrditi što je namjena UI-a i koji podaci su vezani i korisni za tu namjenu. U ovome je radu namjena predvidjeti prosječnu cijenu nekretnine u SAD-u. Za ovo je potrebno prvo odrediti ovisan podatak[6] te neovisne podatke. Ovisan podatak u našem slučaju bit će prosječna cijena nekretnine, a neovisni podaci bit će stopa nezaposlenosti, prosječna cijena prihoda, CPI[7] i populacija.

Zillow nudi svoj „*Zillow Home Value Index*“ (*ZHVI*) u kojem se nalaze cijene prosječne nekretnine po mjesečnoj bazi od 2000. godine pa do 2025. godine. Problem je što *ZHVI* zapisuje cijene nekretnina po gradu u SAD-u, a ne samo jednu prosječnu vrijednost za cijeli SAD.

[6] Podatak kod kojeg se traže promjene uzorkovane neovisnim podacima.
[7] *Consumer Price Index*, mjera prosječne promjene cijena tijekom vremena koje plaćaju urbani potrošači za tržišnu košaricu roba i usluga široke potrošnje. (U.S. Bureau Of Labor, 2025)

Tablica 1 ZHVI tablica podataka

ID	Size	Name	Type	State	County	1/31/2000	2/29/2000
6181	0	New York	city	NY	Queens County	232864.1	234103.3
12447	1	Los Angeles	city	CA	Los Angeles County	216965.4	217293.9
39051	2	Houston	city	TX	Harris County	100425.3	100399.8
17426	3	Chicago	city	IL	Cook County	142060.4	142098.4
6915	4	San Antonio	city	TX	Bexar County	96888.75	96978.7
13271	5	Philadelphia	city	PA	Philadelphia County	59556.58	59680.02
40326	6	Phoenix	city	AZ	Maricopa County	113349	113624.4
18959	7	Las Vegas	city	NV	Clark County	149570.8	149544.3

Kaggle[8] nudi bazu stopa nezaposlenosti po zemlji počevši od 1991. do 2021. godine. Mjerenja su se za ovu bazu odvijala svaku godinu.

Tablica 2 Kaggle tablica nezaposlenosti

Country Name	Country	1991	1992	1993	1994	1995
Africa Eastern and Southern	AFE	7.8	7.84	7.85	7.84	7.83
Afghanistan	AFG	10.65	10.82	10.72	10.73	11.18
Africa Western and Central	AFW	4.42	4.53	4.55	4.54	4.53
Angola	AGO	4.21	4.21	4.23	4.16	4.11
Albania	ALB	10.31	30.01	25.26	20.84	14.61
Arab World	ARB	11.62	12.12	12.86	12.95	13.32
United Arab Emirates	ARE	1.87	1.84	1.85	1.81	1.8

Federalna rezervna banka St. Louisa na svojoj stranici s ekonomskim podacima nudi bazu prosječnih prihoda po kućanstvu u SAD-u. Ova baza je ažurirana na mjesečnoj bazi od 1984. do 2023. godine.

Tablica 3 FRED tablica prihoda

observation_date	MedianHouseholdIncome
1/1/1984	58930
1/1/1985	60050
1/1/1986	62280

[8] Kaggle.com – stranica na kojoj se mogu pronaći javno dostupni skupovi podataka za umjetnu inteligenciju i statistiku.

1/1/1987	63060
1/1/1988	63530
1/1/1989	64610
1/1/1990	63830
1/1/1991	61960
1/1/1992	61450
1/1/1993	61150
1/1/1994	61800

Na istoj se stranici može i naći baza populacije u SAD-u, koja je također ažurirana mjesečno, ali mjerenja počinju od 1959. godine i traju do 2025 godine.

observation_date	POPTHM
1/1/1959	175818
2/1/1959	176044
3/1/1959	176274
4/1/1959	176503
5/1/1959	176723
6/1/1959	176954
7/1/1959	177208
8/1/1959	177479
9/1/1959	177755
10/1/1959	178026

Tablica 4 FRED tablica populacije

Priprema podataka

Priprema podataka može se raditi ručno, no to je dugotrajan i nepotreban proces. Umjesto toga, najčešće se koristi Python te biblioteke za python. Najkorištenije su Pandas, Numpy i Sklearn.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

housing_df = pd.read_csv('../data/raw/HOUSEPRICES.csv')
income_df = pd.read_csv('../data/raw/MEDIANINCOME.csv')
population_df = pd.read_csv('../data/raw/POPULATION.csv')
cpi_df = pd.read_csv('../data/raw/CPI.csv')
unemployment_df = pd.read_csv('../data/raw/UNEMPLOYMENT.csv')
date_columns = housing_df.columns[8:]
```

Ovo je programski kod za učitavanje svih potrebnih biblioteka za pripremu podataka, kao i svih prijašnje skinutih podataka. **Date_columns** je dio tablice podataka cijena nekretnina koji sadrži same cijene po mjesečnoj bazi.

Prvo se treba pripremiti tablica cijena nekretnina budući da je to ovisna varijabla.

```
housing_melted = pd.melt(
    housing_df,
    id_vars=['RegionID', 'RegionName', 'RegionType', 'StateName', 'State'],
    value_vars=date_columns,
    var_name='Date',
    value_name='Price'
)
housing_melted['Date'] = pd.to_datetime(housing_melted['Date'])

housing_avg = housing_melted.groupby('Date')['Price'].mean().reset_index()
housing_avg = housing_avg.rename(columns={'Price': 'AvgHousePrice'})

housing_avg.set_index('Date', inplace=True)
```

Pd.melt je funkcija koja preoblikuje podatke u format kod kojeg je svaki redak tablice jedno mjerenje u vremenu. Zatim, metoda **to_datetime** mijenja tip varijable „Date“ u vremensku varijablu umjesto da ostaje kao niz karaktera.

Za ovaj rad potrebna je prosječna cijena nekretnine u cijelome SAD-u tako da treba izračunati aritmetičku sredinu cijena iz tablice **housing_melted**. Ovo se postiže metodom **mean()**, no treba i ponovo postaviti index tablice nakon ovog mijenjanja, koristeći metodu **reset_index()**. Radi razumijevanja unutar tablice također se promijeni ime stupca ‘Price’ u ‘AvgHousePrice’. Konačno postavlja se index tablice na stupac date i svojstvom **inplace** se odredi da se tablica samo modificira umjesto stvaranja nove.

Konačna tablica izgleda ovako:

Date	AvgHousePrice
2000-01-31	143033.555458
2000-02-29	143158.534386
2000-03-31	143410.996369
2000-04-30	144223.363551
2000-05-31	145047.533509
...	...
2024-09-30	328734.397923
2024-10-31	329668.738871
2024-11-30	330407.163146
2024-12-31	331391.238354
2025-01-31	332158.971771

Tablica 5 Konačna tablica cijena

Zatim dolazi na red uređivanje prosječnog prihoda, no ovdje je potreban i korak manipulacije vremenskih intervala uz mijenjanje tipa varijable i postavljanja indexa. Budući da je tablica prosječnog prihoda ispunjena mjerenjima na godišnjoj bazi treba ju pretvoriti u mjesečne intervale te za svaki mjesec u godini ispuniti istom vrijednosti.

```
income_df['observation_date'] = pd.to_datetime(income_df['observation_date'])
income_df.set_index('observation_date', inplace=True)
income_df = income_df.resample('MS').ffill()
```

Metodom **resample('MS')** se mijenja sam interval tako da mjerenja budu početkom svakog mjeseca (*Month Start – MS*), a **ffill()** metoda ispunjava novo kreirana mjesta sa zadnjom prošlom ispunjenom vrijednosti.

Tablica CPI je najjednostavnija, treba se samo promijeniti tip varijable i postaviti točan index.

```
cpi_df['observation_date'] = pd.to_datetime(cpi_df['observation_date'])
cpi_df.set_index('observation_date', inplace=True)
```

Za tablicu nezaposlenosti treba se najprije izolirati red podataka za SAD.

```
unemployment_us = unemployment_df[unemployment_df['Country Name'] == 'United States']
```

Nakon ovoga treba maknuti sve podatke osim samih vrijednosti i godina koristeći metodu **iloc[ ].** Budući da je oblik matrice pogrešan jer je svaki stupac mjerenje u vremenu, umjesto da je svaki redak mjerenje, matricu je potrebno i transponirati metodom **.T**.

```
unemployment_years = unemployment_us.iloc[:, 2:].T
```

U idućem koraku treba promijeniti tip varijable u vremenski tip te imenovati stupac.

```
unemployment_years.index = pd.to_datetime(unemployment_years.index, format='%Y')
unemployment_years.columns = ['Unemployment']
```

Tablica nezaposlenosti je također ispunjena na godišnjoj bazi pa i nju treba ispuniti na mjesečnoj bazi, istim programskim kodom kao i za prihod.

```
unemployment_monthly = unemployment_years.resample('MS').ffill()
```

Glavna varijabla, prosječna cijena nekretnina, je mjerena na kraju mjeseca pa sve ostale podatke treba prilagoditi tom formatu mjerenja kako bi neuronskoj mreži bilo lakše „učiti“.

```
income_df = income_df.resample('ME').last()
population_df = population_df.resample('ME').last()
cpi_df = cpi_df.resample('ME').last()
unemployment_monthly = unemployment_monthly.resample('ME').last()
```

Sljedeće, trebaju se odrediti zajednički početci i krajevi mjerenja po podatku kako bi završna tablica bila kompletno ispunjena.

```
income_df = income_df.resample('ME').last()
population_df = population_df.resample('ME').last()
cpi_df = cpi_df.resample('ME').last()
unemployment_monthly = unemployment_monthly.resample('ME').last()

common_start = '2000-01-31'

common_end = min(
    housing_avg.index.max(),
    income_df.index.max(),
    population_df.index.max(),
    cpi_df.index.max(),
    unemployment_monthly.index.max()
)

housing_filtered = housing_avg[common_start:common_end]
income_filtered = income_df[common_start:common_end]
population_filtered = population_df[common_start:common_end]
cpi_filtered = cpi_df[common_start:common_end]
unemployment_filtered = unemployment_monthly[common_start:common_end]
```

Idući korak je spajanje svih podataka u jednu tablicu, za ovo se koristi metoda **concat()**.

```
merged_data = pd.concat([
    housing_filtered,
    income_filtered,
    population_filtered['POPTHM'].rename('Population'),
    cpi_filtered['CPI'],
    unemployment_filtered
], axis=1)
```

Ispune se sve nepostojeće podatke sa zadnjom poznatom vrijednosti[9].

```
merged_data.ffill(inplace=True)
```

Konačno, kako bi se maksimizirao dobitak iz sakupljenih podataka, izvodi se *Feature engineering*. Ovo je postupak stvaranja novih svojstva iz sakupljenih podataka kako bi neuronska mreža bolje „učila“. Prvo se stvaraju svojstva kašnjenja[10] kako bi model dobio dojam sezonalnosti u podacima. Kreiraju se ovako:

[9] Ovo je dovoljno dobra metoda popunjavanja nedostajećih podataka za ekonomske podatke, naročito na kratkim periodima.
[10] Svojstvo koje u svakom vremenskom koraku sadrži proizvoljnu prošlu vrijednost varijable.

```
for lag in range(1, 13):
    merged_data[f'Price_Lag_{lag}'] = merged_data['AvgHousePrice'].shift(lag)
```

Iduća svojstva su pomične aritmetičke sredine. Kreirane su 3 - jedna za prethodna 3 mjeseca, jedna za prethodnih 6 mjeseci i jedna za prethodnih 12 mjeseci. Ovo su svojstva koja omogućuju modelu da dobije širi dojam promjena vrijednosti.

```
merged_data['Price_3M_MA'] = merged_data['AvgHousePrice'].rolling(window=3).mean()
merged_data['Price_6M_MA'] = merged_data['AvgHousePrice'].rolling(window=6).mean()
merged_data['Price_12M_MA'] = merged_data['AvgHousePrice'].rolling(window=12).mean()
```

Također, dobro je za model kreirati i indexe mjeseca i godina kako bi lakše usvojio sezonalnost i godišnje trendove.

```
merged_data['Month'] = merged_data.index.month
merged_data['Year'] = merged_data.index.year
```

Model trenutačno dobiva dojam linearne povezanosti između mjeseci i ovo je loše jer je 12. mjesec jako blizu 1. mjesecu, a model to ovako ne razumije, također, model može interpretirati indexe mjeseca kao rangiranje važnosti. Zbog ovoga, koristi se metoda nazvana jednojedinično kodiranje. Za ovaj specifičan slučaj kreiraju se *dummy* varijable koje su binarnog tipa. Kreira se 11 tih *dummy* varijabli (1. mjesec se izostavi[11]). Ako je za određeno mjerenje istinito da je 3. mjesec onda će odgovarajuća varijabla (Month_3) biti istinita (1).

```
month_dummies = pd.get_dummies(merged_data['Month'], prefix='Month', drop_first=True)
merged_data = pd.concat([merged_data, month_dummies], axis=1)
```

Kako bi model dobio jasni dojam o dugotrajnim promjenama uvode se i svojstva godišnjih promjena za najvažnije podatke te podatke koji se najviše mijenjaju.

```
merged_data['YoY_Price_Change'] = merged_data['AvgHousePrice'].pct_change(12)
merged_data['YoY_Population_Change'] = merged_data['Population'].pct_change(12)
merged_data['YoY_Income_Change'] = merged_data['MedianHouseholdIncome'].pct_change(12)
```

Konačno, izbrišu se nedostajući podaci.

```
merged_data_clean = merged_data.dropna()
```

[11] Zbog redundancije, ako su svi ostali lažne varijable 0 onda model zna da je prvi mjesec.

Ovo je sada finalna tablica za model, no problem je što sva svojstva neće jednako pridonijeti modelu budući da su, po svojoj prirodi, neka svojstva puno veći brojevi nego druga. Kako bi se ovo ublažilo, sva se svojstva postavljaju u mjerilu od 0 do 1. Formula za ovo glasi:

$$X_{skalirano} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

```
features = [
    'AvgHousePrice', 'MedianHouseholdIncome', 'Population', 'CPI', 'Unemployment',
    'Price_Lag_1', 'Price_Lag_2', 'Price_Lag_3', 'Price_Lag_6', 'Price_Lag_12',
    'Price_3M_MA', 'Price_6M_MA', 'Price_12M_MA',
    'YoY_Price_Change', 'YoY_Population_Change', 'YoY_Income_Change'
] + [col for col in merged_data.columns if col.startswith('Month_')]
```

```
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(merged_data_clean[features])
scaled_df = pd.DataFrame(scaled_data, columns=features, index=merged_data_clean.index)
```

	AvgHousePrice	MedianHouseholdIncome	Population	CPI	\
2001-01-31	154024.146023	68870	283960	171.700	
2001-02-28	155039.976457	68870	284166	172.400	
2001-03-31	156035.123228	68870	284380	172.600	
2001-04-30	157078.065681	68870	284602	173.500	
2001-05-31	158057.794671	68870	284834	174.400	
...	...	...	...	...	
2020-09-30	242279.531817	79560	332017	254.004	
2020-10-31	245963.696617	79560	332092	254.076	
2020-11-30	250154.083684	79560	332129	253.826	
2020-12-31	254116.736227	79560	332113	254.081	
2021-01-31	257754.286018	79260	332058	255.296	

	Unemployment	Price_Lag_1	Price_Lag_2	Price_Lag_3	\
2001-01-31	4.73	153011.794451	151811.849953	150668.331250	
2001-02-28	4.73	154024.146023	153011.794451	151811.849953	
2001-03-31	4.73	155039.976457	154024.146023	153011.794451	
2001-04-30	4.73	156035.123228	155039.976457	154024.146023	
2001-05-31	4.73	157078.065681	156035.123228	155039.976457	
...	...	...	...	...	
2020-09-30	8.05	239276.106022	237670.200687	237169.220088	
2020-10-31	8.05	242279.531817	239276.106022	237670.200687	
2020-11-30	8.05	245963.696617	242279.531817	239276.106022	
2020-12-31	8.05	250154.083684	245963.696617	242279.531817	
2021-01-31	5.46	254116.736227	250154.083684	245963.696617	

	Price_Lag_4	Price_Lag_5	...	Month_6	Month_7	Month_8	\
2001-01-31	149495.295567	148296.896927	...	False	False	False	
2001-02-28	150668.331250	149495.295567	...	False	False	False	
2001-03-31	151811.849953	150668.331250	...	False	False	False	
2001-04-30	153011.794451	151811.849953	...	False	False	False	
2001-05-31	154024.146023	153011.794451	...	False	False	False	
...	...	...	...	...	...	...	
2020-09-30	236862.991167	236017.779552	...	False	False	False	
2020-10-31	237169.220088	236862.991167	...	False	False	False	
2020-11-30	237670.200687	237169.220088	...	False	False	False	
2020-12-31	239276.106022	237670.200687	...	False	False	False	
2021-01-31	242279.531817	239276.106022	...	False	False	False	

Tablica 6 Ne-skalirani podaci

	AvgHousePrice	MedianHouseholdIncome	Population	CPI	\
2001-01-31	0.000000	0.202327	0.000000	0.000000	
2001-02-28	0.009793	0.202327	0.004277	0.008374	
2001-03-31	0.019387	0.202327	0.008719	0.010766	
2001-04-30	0.029441	0.202327	0.013328	0.021532	
2001-05-31	0.038886	0.202327	0.018144	0.032298	
...	...	...	...	...	
2020-09-30	0.850817	0.893342	0.997675	0.984545	
2020-10-31	0.886334	0.893342	0.999232	0.985406	
2020-11-30	0.926731	0.893342	1.000000	0.982415	
2020-12-31	0.964933	0.893342	0.999668	0.985466	
2021-01-31	1.000000	0.873950	0.998526	1.000000	

	Unemployment	Price_Lag_1	Price_Lag_2	Price_Lag_3	Price_Lag_6	\
2001-01-31	0.177852	0.000000	0.000000	0.000000	0.000000	
2001-02-28	0.177852	0.010013	0.012202	0.012000	0.011715	
2001-03-31	0.177852	0.020060	0.022496	0.024592	0.024382	
2001-04-30	0.177852	0.029903	0.032825	0.035215	0.036781	
2001-05-31	0.177852	0.040218	0.042945	0.045875	0.048868	
...	...	...	...	...	...	
2020-09-30	0.734899	0.853216	0.873057	0.907713	0.923489	
2020-10-31	0.734899	0.882922	0.889387	0.912971	0.938944	
2020-11-30	0.734899	0.919361	0.919927	0.929822	0.947878	
2020-12-31	0.734899	0.960807	0.957390	0.961340	0.951115	
2021-01-31	0.300336	1.000000	1.000000	1.000000	0.956411	

	Price_Lag_12	...	Month_3	Month_4	Month_5	Month_6	Month_7	\
2001-01-31	0.000000	...	0.0	0.0	0.0	0.0	0.0	
2001-02-28	0.001265	...	0.0	0.0	0.0	0.0	0.0	
2001-03-31	0.003822	...	1.0	0.0	0.0	0.0	0.0	
2001-04-30	0.012047	...	0.0	1.0	0.0	0.0	0.0	
2001-05-31	0.020393	...	0.0	0.0	1.0	0.0	0.0	
...	...	...	...	...	...	...	...	
2020-09-30	0.862361	...	0.0	0.0	0.0	0.0	0.0	
2020-10-31	0.869369	...	0.0	0.0	0.0	0.0	0.0	
2020-11-30	0.878832	...	0.0	0.0	0.0	0.0	0.0	
2020-12-31	0.890366	...	0.0	0.0	0.0	0.0	0.0	
2021-01-31	0.901252	...	0.0	0.0	0.0	0.0	0.0	

	Month_8	Month_9	Month_10	Month_11	Month_12
2001-01-31	0.0	0.0	0.0	0.0	0.0
2001-02-28	0.0	0.0	0.0	0.0	0.0

Tablica 7 Skalirani podaci

Završni korak prije samog stvaranja modela je cijepanje krajnjih podataka u sekvence za treniranje, validaciju i testiranje. Prvo se treba definirati kako se stvara sekvenca, to radi sljedeća funkcija:

```
def create_sequences(data, seq_length=12):
    X, y = [], []

    target_col = 'AvgHousePrice'
    feature_cols = [col for col in data.columns if col != target_col]

    for i in range(len(data) - seq_length):
        X.append(data[feature_cols].iloc[i:i+seq_length].values)
        y.append(data[target_col].iloc[i+seq_length])

    return np.array(X), np.array(y)
```

Funkcija iz podataka stvara ulazna svojstva za neuronsku mrežu X i ciljane izlazne vrijednosti y. 70 % podataka dijelimo za treniranje, 20 % za validaciju i 10 % za testiranje treniranog modela.

```
train_size = int(len(df) * 0.7)
val_size = int(len(df) * 0.2)
test_size = len(df) - train_size - val_size

train_data = df[:train_size]
val_data = df[train_size:train_size+val_size]
test_data = df[train_size+val_size:]
```

Sad se odcijepljene podatke pretvara u sekvence.

```
seq_length = 12
X_train, y_train = create_sequences(train_data, seq_length)
X_val, y_val = create_sequences(val_data, seq_length)
X_test, y_test = create_sequences(test_data, seq_length)
```

Model

Stvaranje modela radi se s pomoću biblioteke Tensorflow i Keras. Programski kod za stvaranje modela je:

```
def build_lstm_model(input_shape):

    from tensorflow.keras.layers import Input

    model = Sequential()

    model.add(Input(shape=input_shape))

    model.add(LSTM(64, activation='relu', return_sequences=True))
    model.add(Dropout(0.2))

    model.add(LSTM(32, activation='relu'))
    model.add(Dropout(0.2))

    model.add(Dense(1))

    model.compile(
        optimizer='adam',
        loss='mse',
        metrics=['mae']
    )

    return model

input_shape = (X_train.shape[1], X_train.shape[2])

model = build_lstm_model(input_shape)
```

Ovo je funkcija za stvaranje modela, kao parametar uzima oblik ulaznih podataka. Ulazni podaci su lista koja u sebi sadrži broj vremenskih koraka po svakoj sekvenci te broj svojstava (ulaznih podataka).

Objašnjenje programskog koda:

from tensorflow.keras.layers import Input – Ovako se „unosi“ programski kod za stvaranje ulaznog sloja

model = Sequential() – Stvara sekvencijalni model kod kojeg su slojevi postavljeni u linearnu hrpu stvarajući model

model.add(Input(shape=input_shape)) – Na model dodaje ulazni sloj oblika parametra input_shape

model.add(LSTM(64, activation='relu', return_sequences=True)) – Dodaje na model prvi LSTM sloj sa 64 LSTM jedinice, definira aktivacijsku funkciju ReLU[12] i omogućuje vraćanje cijele sekvencije izlaza kako bi idući sloj mogao obraditi sekvencijalne podatke

model.add(Dropout(0.2)) – Dodaje sloj izbacivanja sa stopom 20 %

model.add(LSTM(32, activation='relu')) – Dodaje se još jedan LSTM sloj s 32 LSTM jedinice te određuje aktivacijsku funkciju ReLU, kao i u prijašnjem LSTM sloju. Neuroni ovog sloja daju izlaz jednog vektora po sekvenci

model.add(Dropout(0.2)) – Dodaje sloj izbacivanja sa stopom 20 %

model.add(Dense(1)) – Dodaje izlazni sloj s jednim neuronom, aktivacijska funkcija po zadanim postavkama je linearna

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

[12] *Rectified Linear Unit* – definirana kao ne-negativan dio argumenta.

Ovaj blok programskog koda konfigurira model za treniranje. Postavlja optimizacijski algoritam Adam[13], određuje funkciju grešaka MSE[14]. Konačno, određuje koje metrike će pratiti, u ovom slučaju je to MAE[15].

return model – Vraća kompletno strukturiran model

input_shape = (X_train.shape[1], X_train.shape[2]) – Definira se ulazni oblik u odnosu na podatke za treniranje

model = build_lstm_model(input_shape) – Poziva se funkcija te stvara LSTM model

Ovo su podaci o stvorenom modelu:

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 12, 64)	23,296
dropout (Dropout)	(None, 12, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 1)	33

Total params: 35,745 (139.63 KB)

Trainable params: 35,745 (139.63 KB)

Non-trainable params: 0 (0.00 B)

Slika 3 Arhitektura modela

[13] *ADaptive Moment estimation*, vrlo često korišteni optimizacijski algoritam koji prilagođava stopu učenja tijekom treniranja.
[14] *Mean Square Error*, često korištena funkcija za računanje gubitaka.
[15] *Mean Absolute Error*, apsolutna greška.

Svojstvo *Output Shape* je lista podataka formata: veličina serije[16], duljina sekvence, broj neurona. Ako je veličina serije „None“ to znači da je fleksibilna varijabla. Param # je broj parametara po sloju.

Treniranje modela

```
callbacks = [
    EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5, min_lr=0.0001),
    ModelCheckpoint('house_price_lstm_model.keras', save_best_only=True)
]

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=64,
    callbacks=callbacks,
    verbose=1
)
```

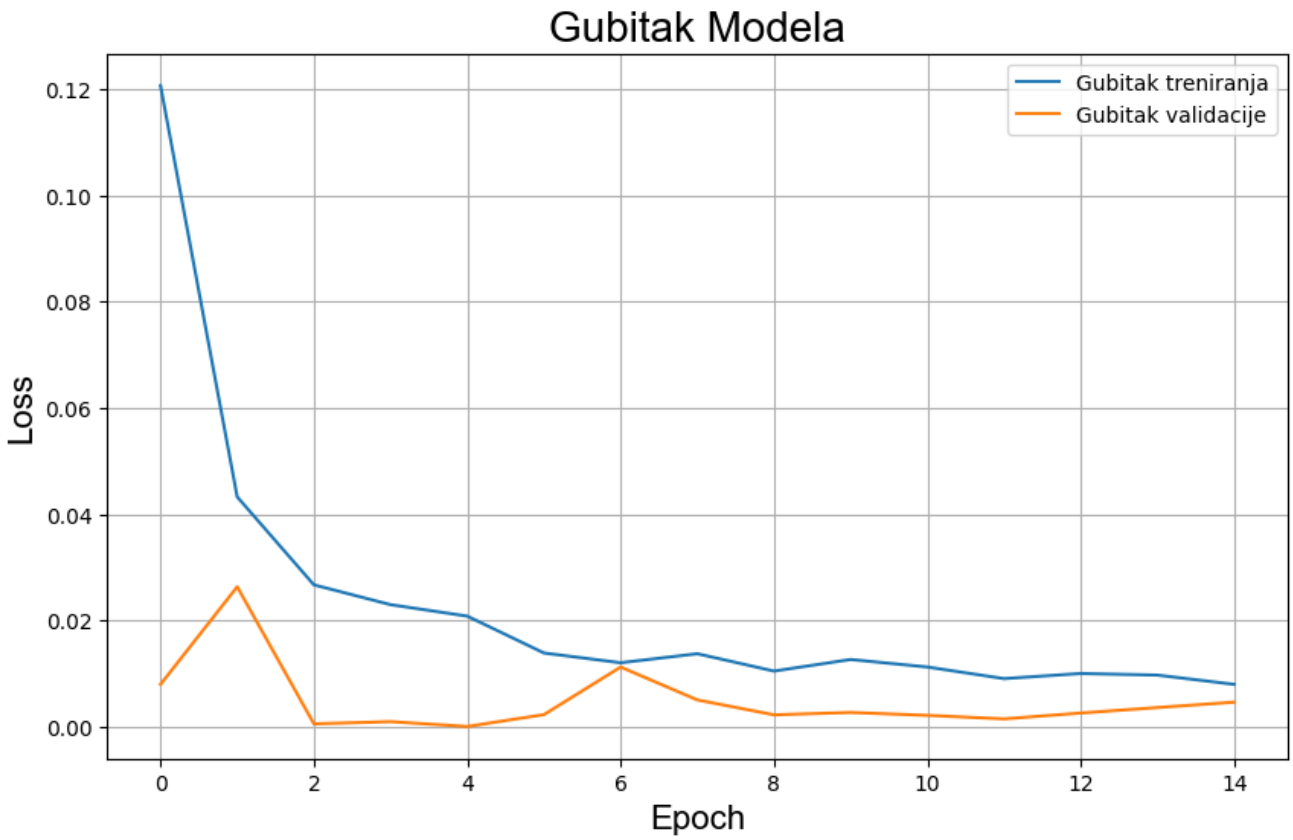
Callback su metode koje automatiziraju neke postupke kao: zaustavljanje treniranja, prilagođavanje stope učenja i spremanje modela. **EarlyStopping** je funkcija koja zaustavi treniranje ako se greška validacije ne poboljša nakon 10 epoha i vrati najbolje postignute težine. **ReduceLROnPlateau** je metoda koja smanjuje stopu učenja ako se greška validacije ne poboljša nakon 5 epoha. **ModelCheckpoint** spremi model kad god se poveća greška validacije, ovako se osigura da je spremljen najbolji model.

Samo treniranje je izvedeno blokom programskog koda nakon. Ovdje su definirani podaci za treniranje i validaciju te neki drugi hiper-parametri kao broj epoha i veličina serije.

[16] Hiper-parametar koji diktira koliko će uzoraka podataka model obraditi prije nego što mijenja parametre.

Rezultati

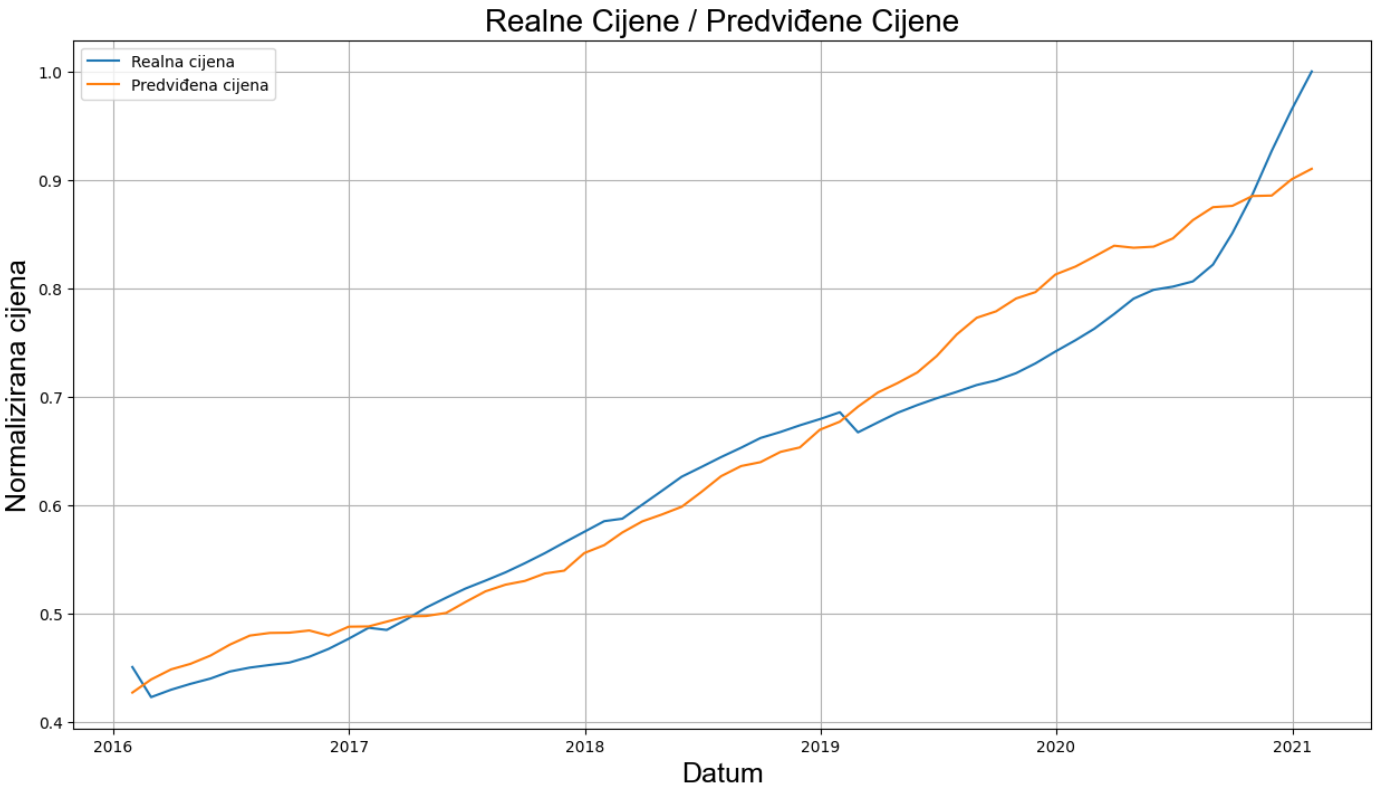
Iduće je graf funkcije grešaka po epohi treniranja:



Slika 4 Graf gubitaka modela po epohi treniranja i validacije

Čitljivo je s grafa da je u prve 3 epohe gubitak naglo pao, što znači da je model brzo naučio ključne obrasce u podacima. Do 3. epohe gubitak pri validaciji čak padne ispod gubitka kod treniranja i dostiže jako male vrijednosti što sugerira da model dobro generalizira i izbjegava prenaučenost. Manje fluktuacije nakon naglog pada su normalne i opet ukazuju na to da model izbjegava prenaučenost te se može naslutiti da su slojevi ispadanja učinkoviti. Uspjeh modela

se mjeri na podacima za testiranje, tj. podacima koje još nikad nije koristio. Iduće je graf koji prikazuje koliko je model precizan na tim podacima.



Slika 5 Graf predviđanja modela i stvarnih cijena iz seta podataka za testiranje

Model je konzistentno pratio realne cijene nekretnine u ranim mjesecima.

Oko siječnja 2020. sijeku se predviđanja i stvarne vrijednosti. Ovo sugerira da model nije uspio predvidjeti stagnaciju cijena koja je slijedila. Nakon srpnja, model na kratko precjenjuje vrijednosti dok stvarne cijene stagniraju. Model je očekivao uzastopni rast, dok se tržište stabiliziralo privremeno.

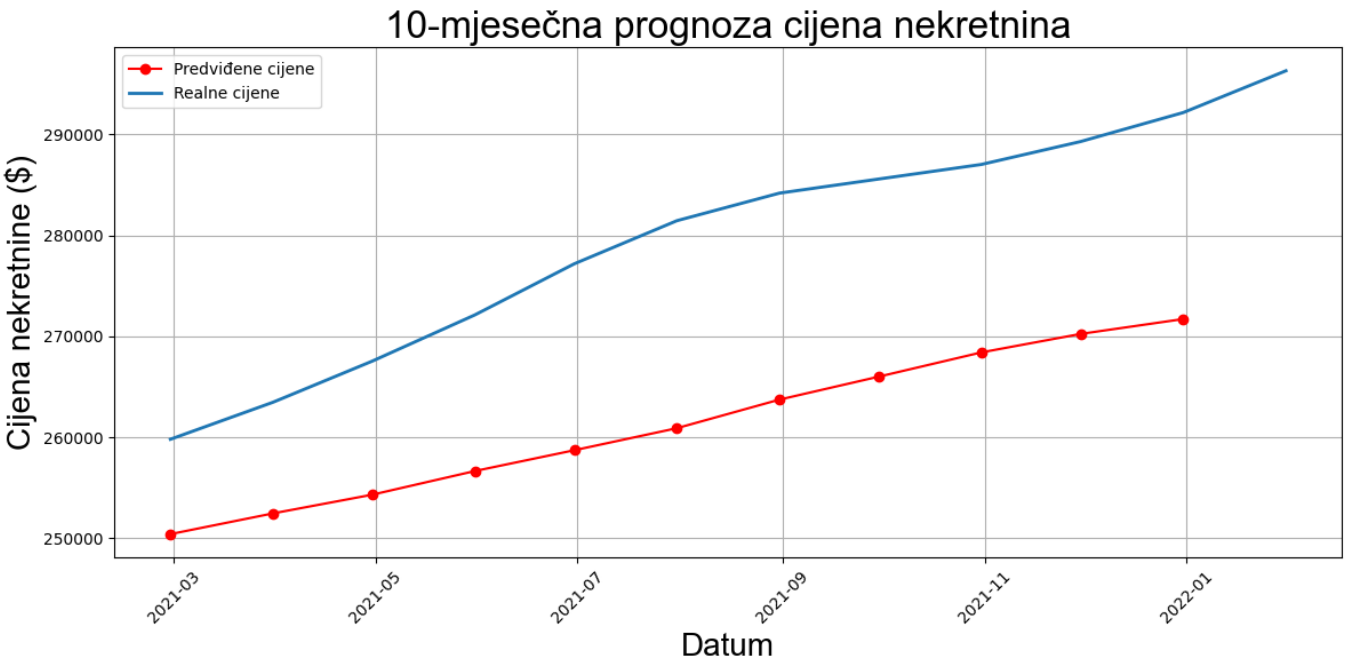
Linije se ponovo susreću oko srpnja 2020. te slijedi značajno podcjenjivanje. Značajno odstupanje se događa u posljednjim mjesecima jer model nije uspio uhvatiti dramatično ubzranje rasta cijena kuća krajem 2020. i početkom 2021. godine.

Iz ovoga grafa možemo pretpostaviti:

1. Model ima poteškoće s predviđanjem naglih promjena tržišta.
2. Ima bolju funkcionalnost tijekom razdoblja stalnih promjena.

Geopolitičko stanje predstavlja ključne vanjske varijable koje snažno utječu na ekonomske trendove. Iako su standardni ekonomski pokazatelji kao CPI, nezaposlenost ... kvantificirani i inkorporirani, nepredvidljivi događaji poput financijskih kriza i globalne napetosti se teško modeliraju numerički.

Konačno, idući grafa predstavlja predviđanja modela od 3. mjeseca 2021. godine do 1. mjeseca 2022. godine te stvarne izmjerene vrijednosti. Ovi podaci su izvan granica podataka treniranja modela i vidimo da model shvaća opći trend kretanja, ali da ima grešaka kod predviđanja konkretnih vrijednosti.



Slika 6 Graf predviđanja modela i realnih cijena izvan raspona seta podataka na kojemu je model testiran i treniran

Zaključak

U radu je detaljno prikazan proces primjene umjetne inteligencije, specifično LSTM mreža, u ekonomiji, za predviđanje ekonomskih faktora. Ključni su koraci bili: prikupljanje i obrađivanje važnih ekonomskih podataka (CPI, populacija, nezaposlenost...), inženjering svojstava (stvaranje kašnjenja, pomični prosjeci...) te stvaranje i treniranje modela. Srž modela su 2 LSTM sloja uz koja se nalaze i 2 sloja za izbacivanje kako bi se spriječila prenaučenost. Optimizacijski algoritam je bio Adam algoritam i korištene su brojne *callback* funkcije za efikasnije treniranje. Rezultati pokazuju manje greške, naročito pri početku, ali dobru generalizaciju trendova i validne performanse u periodima stalnih promjena. Modeli umjetne inteligencije se uvijek mogu poboljšati i nikad nije kraj optimizacije, tako se i model u ovom radu može poboljšati. Prvi korak

za unaprjeđenje bilo kojeg modela jest proširenje skupa ulaznih varijabli. Najočitije bi bilo dodati još neke ključne podatke kao stopu inflacije i kamatnu stopu, ali i uključivanje socio-političkih podataka bi drastično poboljšalo model. Koristeći neke vanjske indexe ili neke druge metode za kvantificiranje podataka kao npr. politički rizici i opća napetost, pomoglo bi ovom modelu da bolje razumije obrasce u stvarnom svijetu. UI je grana koja se svaki dan mijenja te se neprestano stvaraju nove arhitekture. Korištenje neke naprednije arhitekture kao Transformere moglo bi dodatno poboljšati rezultate. Konačno, model bi se mogao podložiti optimizaciji hiper-parametara, kao što su broj slojeva, jedinica, stopa izbacivanja... koristeći algoritme za isto.

Literatura

Armstrong, Robert. „AI can pick stocks better than you can“. 11. lipnja 2024. https://www.ft.com/content/a680e7a2-2799-4fc9-ace9-b01770b260df?utm_source=chatgpt.com, (pristupljeno: 10.2.2025.)

Banerji, Priyanka Dr.. „The Role of Artificial Intelligence (AI) in Economics“. 20. siječnja 2025. https://www.ncuindia.edu/the-role-of-artificial-intelligence-ai-in-economics/, (pristupljeno: 2.2.2025.)

Bishop, Christopher M. 2006. „Pattern Recognition and Machine Learning“. *Springer Science* i *Business Media LLC.* New York, NY

Boppiniti, Sai Teja. „Artificial Intelligence In Financial Markets: Algorithms And Applications“. *International Journal Of Innovations In Engineering Research And Technology*. 8, 1(2021). 118-127

Chen, Xiaochen; Wei, Lai; Xu, Jiaxin. „House Price Prediction Using LSTM“. *The Hong Kong University of Science and Technology. 2017*

Dutta, Sanjay. „Number of Neurons per Hidden Layer in Neural Networks: A Guide“. 5. lipnja 2024. https://medium.com/@sanjay_dutta/number-of-neurons-per-hidden-layer-in-neural-networks-a-guide-106fea04fbfe, (pristupljeno: 15.1.2025.)

Dizikes, Peter. „What do we know about the economics of AI“? 6. prosinca 2024. https://news.mit.edu/2024/what-do-we-know-about-economics-ai-1206, (pristupljeno: 25.1.2025.)

Hardesty, Larry. „Explained: Neural networks“. 14. travnja 2017. https://news.mit.edu/2017/explained-neural-networks-deep-learning-0414, (pristupljeno: 9.1.2025.)

Hochreiter, Sepp; Schmidhuber, Jurgen. „Long short-term memory“. *Neural Computation. 9, 1(1997). 1735-1780*

Hochreiter, Sepp. „The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions“. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems. 6, 2(1998). 107-116*

Kelleher, John D. 2019. „Deep learning“. *The MIT Press.* Cambridge, MA

Krstičević, Mia. 2024. Klasifikacija Respiratornih Bolesti Na Temelju Zvučnih Zapisa Auskultacije Pluća Korištenjem Metoda Strojnog Učenja. Završni rad. Sveučilište u Zagrebu, Fakultet elektrotehnike i računarstva Zagreb. Zagreb. 29 str.

Russell, Stuart J. i Norvig, Peter. 2021. „Artificial Intelligence: A Modern Approach“. *Pearson Education Inc.* Hoboken, NJ

Sharma, Avinash. Understanding Activation Functions in Neural Networks. 30. ožujka 2017. https://medium.com/the-theory-of-everything/understanding-activation-functions-in-neural-networks-9491262884e0, (pristupljeno: 3.1.2025.)

Sherstinsky, Alex. „Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) network“. *Physica D: Nonlinear Phenomena. 404, 2020.*

Srivastava, Nitish; Hinton, Geoffrey; Krizhevsky, Alex; Sutskever, Ilya; Salakhutdinov, Ruslan. „Dropout: A Simple Way to Prevent Neural Networks from Overfitting“. *The Journal of Machine Learning Research*. 15, 1(2014). 1929 – 1958

Trafton, Anne. „Study urges caution when comparing neural networks to the brain“. 2. studenog 2022. https://news.mit.edu/2022/neural-networks-brain-function-1102, (pristupljeno: 10.1.2025.)

Wang, Jia; Sun, Tong; Liu, Benyuan; Cao, Yu; Wang, Degang. „Financial Markets Prediction with Deep Learning“. *17. IEEE International Conference on Machine Learning and Applications (ICMLA).* 2018. 97-104

Consumer Price Index. https://www.bls.gov/cpi/, (pristupljeno: 20.12.2024.)

Gradient descent. https://www.khanacademy.org/math/multivariable-calculus/applications-of-multivariable-derivatives/optimizing-multivariable-functions/a/what-is-gradient-descent, (pristupljeno: 10.2.2025)